



SE431 SOFTWARE QUALITY ASSURANCE

MicroLend

Community Microfinance Loan Servicing and Delinquency Engine

MILESTONE 1 · WEEK 6

Quality Requirements, RTM & SQA Plan

TEAM FABLE

Muhammad Ibrahim

TEAM LEAD

2023446 · SQAP outline · risk register

Hassan Khalid

2023242 · Quality requirements · RTM

Tughral Hussain

2023532 · Standards · Cost of Quality

INSTRUCTOR

Dr. Mian Muaz Razaq

REPOSITORY

github.com/MicroLend-SE423/microlend

Contents

1	Quality Requirements (ISO/IEC 25010)	3
2	Requirements Traceability Matrix — Version 1	5
3	SQA Plan — Scope and Outline (IEEE 730)	9
4	Cost of Quality and Quality-Culture Statement	11
5	Standards Awareness	12
6	Initial Risk Register	13
7	Summary of M1 Quality Position	15
Annex A — AI Usage Log		16
	Declaration for Milestone 1	17
	Team rules for AI use	17

Companion document: <docs/M1-SE423-Proposal-000-Pattern-Plan.md> — the construction-side proposal this quality plan applies to.

1 Quality Requirements (ISO/IEC 25010)

1.1 Which characteristics matter for this system, and which do not

ISO/IEC 25010:2011, the version used in this course, defines eight product quality characteristics. MicroLend's product risk is **financial, not safety-related**, and it is an internal back-office tool with a handful of users, which determines where quality effort is spent:

Characteristic	Relevance to MicroLend	Why
Functional suitability	Critical	A schedule or allocation that is arithmetically wrong corrupts every downstream figure — balances, penalties, classification, provisioning, reports
Reliability	Critical	A failure part-way through posting must not leave a half-written ledger; the end-of-day engine must survive a task dying
Security (integrity)	High	A ledger that can be altered is not an audit record. Integrity, not confidentiality, is the sub-characteristic that matters
Maintainability	High	The system is assessed across three milestones by three developers who must each work in code the others wrote
Performance efficiency	Moderate	The end-of-day run must finish within an overnight window. Nothing else is time-critical
Usability	Low	A small internal web interface used by trained branch staff on a local machine. Learnability is not a project risk
Compatibility	Low	Interoperability only at the bank-statement import (FR-20) and the CSV export (FR-32), both verified functionally; no co-existence requirement
Portability	Low	Adaptability only between members' Windows laptops and the Linux CI runner, verified by every CI build; the system is never deployed

No characteristic is quietly omitted: compatibility and portability are rated **Low**, with the reason stated.

1.2 The quality requirements

Each requirement is tied to one characteristic, states a measurable target, and names one of the four verification methods — **inspection, analysis, demonstration, test**.

ID	Requirement	Characteristic	Sub-characteristic	Measurable target	Verification method and evidence
QR-01	End-of-day processing of 10,000 active loans shall complete within 20 seconds on the demo machine using 4 threads	Performance efficiency	Time behaviour	Median wall-clock over 5 consecutive runs ≤ 20 s; no single run > 25 s	Test — timed JUnit harness over a generated 10,000-loan dataset; timings attached to each milestone report

QR-02	Schedule generation shall reproduce the hand-computed reference schedules exactly, at the stated monetary scale	Functional suitability	Functional correctness	Zero discrepancy against 30 hand-computed golden schedules (10 per product) at scale 2, <code>HALF_EVEN</code>	Test — golden-file unit tests (<code>GoldenScheduleTest</code>). Allocation correctness is covered separately by the 100% decision-coverage policy on the money path (SQAP §4.4)
QR-03	Any failure during repayment posting shall leave the ledger balanced, with no partial entry	Reliability	Fault tolerance	For 100% of at least 10 injected failure points: $\Sigma \text{ debits} = \Sigma \text{ credits}$, and zero orphan entries	Test — fault injection through a repository stub that throws at each write point; also demonstrated live at M3 as one of the two required failure scenarios
QR-04	Core business-logic methods shall remain within the stated complexity and size limits	Maintainability	Analysability / Modifiability	Mean cyclomatic complexity ≤ 5 per method across the five module packages, computed from JaCoCo's per-method complexity counter; no method above 15 and no source file above 400 lines (the Checkstyle build gate, §6.5 of the SE423 document)	Analysis — static-analysis metric report produced in CI and attached to each milestone
QR-05	A posted ledger entry shall not be alterable by any component; corrections shall occur only as compensating entries	Security	Integrity	Zero mutating members on <code>LedgerEntry</code> ; zero <code>UPDATE</code> or <code>DELETE</code> statements against <code>ledger_entry</code> in the codebase; 100% of corrections implemented as compensating entries	Inspection — an explicit item on the Fagan inspection checklist, supported by a static-analysis rule and a unit test asserting reversal-only correction
QR-06	The end-of-day engine shall produce identical results whether or not repayments are posted concurrently during the run	Reliability	Maturity	Over 50 repeated runs of the concurrency suite: zero lost updates, zero double accruals, identical final balances to the single-threaded reference run	Test — deterministic latch-driven concurrency tests, repeated 50 times by a scheduled CI job added at M2 with the engine itself

1.3 Constraints (not quality requirements)

The standard separates requirements into functional, non-functional (quality) and **constraints**. The following are constraints on how the system is built. They are recorded in SQAP §4.4 rather than presented as quality requirements:

ID	Constraint	Rationale
C-01	Monetary values shall never be represented as <code>double</code> or <code>float</code>	<code>0.1 + 0.2 $\neq$ 0.3</code> in IEEE 754; a fraction of a rupee per instalment accumulates into a ledger that does not balance. Enforced by a Checkstyle rule that fails the build
C-02	<code>BigDecimal</code> values shall be compared with <code>compareTo() == 0</code> , never <code>equals()</code>	<code>equals()</code> also compares scale, so <code>2.50</code> is not equal to <code>2.5</code>
C-03	No cyclic dependencies between module packages	Cycles destroy the ability to test or replace a module in isolation. Enforced from M2 by an ArchUnit test in CI

C-04 Monetary columns shall be stored as integer minor units Same reasoning as C-01, applied at the storage layer

1.4 The quality trade-off

QR-01 (performance) is in tension with QR-03, QR-05 and QR-06 (fault tolerance, integrity, maturity). Per-account locking, immutable value objects, append-only correction and an idempotency key all cost time behaviour: each adds work on the path that QR-01 measures.

The conflict is resolved by **capping the cost rather than removing either attribute** — locks are per account rather than global, so unrelated accounts are computed in parallel, and each lock is held only for that account's own short transaction. The measured cost is recorded in the GQM data at each milestone rather than asserted to be small.

2 Requirements Traceability Matrix — Version 1

2.1 Purpose and how it will be maintained

The RTM supports traceability in **both directions**. Forward — from requirement to code to test — answers “*did we build and verify everything that was asked for?*” and exposes a **test gap**: a requirement implemented but never verified. Backward — from code to requirement — answers “*why does this code exist?*” and exposes **rogue code**: a module traceable to no approved requirement.

Those are the two defects this matrix is audited for at every milestone. At M1 the design and class columns are planned rather than built, so they are marked planned; from M2 they name real artifacts and real test results.

The matrix is maintained at [qa/rtm.md](#) in the repository and versioned with the code, so the state of traceability at any baseline can be reproduced from the tag.

2.2 Functional requirements

Each requirement traces to a rule in the Business Rules Register ([docs/business-rules.md](#)). The three requirements marked “—” (FR-20, FR-31, FR-32) have no business rule because they are interface or operational requirements, not lending rules.

ID	Requirement	Business rule	Module	Design element (planned)	Planned test — level and technique
FR-01	An applicant shall be at least 18 years old at disbursement and no more than 65 at maturity	BR-01, BR-02	origination	AgeRule	Unit — BVA : 17 / 18 at disbursement; 64 / 65 / 66 at maturity
FR-02	An applicant shall hold no more than 2 active loans, counting the one applied for (a loan in arrears is active; a written-off loan is not)	BR-03	origination	ActiveLoanLimitRule	Unit — EP + BVA : 0, 1, 2, 3 existing active loans (2 and 3 rejected)
FR-03	Total monthly instalment burden, including the new loan, shall not exceed 40% of declared monthly income	BR-04	origination	BurdenRatioRule	Unit — BVA : 39.99% / 40.00% / 40.01%
FR-04	An applicant with a prior write-off shall be rejected	BR-05	origination	WriteOffHistoryRule	Unit — EP : no history / written-off / settled late

FR-05	A rejected application shall state a typed reason naming the first rule that failed	BR-08	origination	EligibilityEvaluator , Decision , RejectionReason	Unit — EP : one case per rejection reason
FR-06	Risk grade A/B/C shall be assigned from the burden ratio and repayment history, and shall adjust the rate by a fixed spread and cap the approved amount	BR-09, BR-10	origination	RiskGrade , Decision	Unit — EP + BVA : one case per grade; burden 25.00 / 25.01 / 35.00 / 35.01%; worst past DPD 30 / 31 / 60 / 61
FR-07	The requested amount and tenure shall lie within the product's bands	BR-06, BR-07	origination	ProductBandRule	Unit — BVA : min – 1 / min / max / max + 1, for amount and tenure
FR-08	The system shall generate schedules for flat-rate, reducing-balance and bullet products	BR-11, BR-17	products	LoanProduct and its three subclasses; InterestCalculator and its three calculators	Unit — Golden file : 10 hand-computed schedules per product
FR-09	Instalment dates shall follow the disbursement day-of-month, clamped for short months	BR-12	products	DueDateCalculator	Unit — BVA : disbursement on the 28th, 29th, 30th, 31st; February in leap and non-leap years
FR-10	A grace period of up to 3 months shall defer the first instalment while interest still accrues	BR-13	products	GracePeriodPolicy	Unit — EP + BVA : grace 0 / 1 / 3 / 4 (4 rejected)
FR-11	The instalments of a schedule shall sum exactly to principal plus total interest	BR-14, BR-15	products	ScheduleAssembler , Schedule , Instalment	Unit — Property-based invariant over valid amount/tenure/rate combinations
FR-12	A schedule shall contain exactly <code>tenureMonths</code> instalments	BR-16	products	ScheduleAssembler	Unit — EP : tenure 1 / 6 / 36, with and without grace
FR-13	Repayments shall be applied in the order penalty → fees → interest → principal, oldest instalment first	BR-18, BR-19	ledger	RepaymentAllocator , method <code>allocate()</code> ; Allocation	Unit — Basis path (CC ≈ 11) + EP on payment size
FR-14	A partial payment shall be applied as far as it reaches, leaving the remainder outstanding	BR-20	ledger	RepaymentAllocator	Unit — BVA : 0 / 0.01 / due – 0.01 / exactly due
FR-15	An overpayment shall become an advance credit unless it clears the loan, in which case early settlement applies	BR-21, BR-22	ledger	SettlementPolicy	Unit — BVA : due + 0.01 / exact payoff / payoff + 0.01, per product
FR-16	Every posting shall write balanced debit and credit lines	BR-23	ledger	CoreRepaymentPoster	Unit — Invariant : $\sum \text{debits} = \sum \text{credits}$ after every posting

FR-17	A posted ledger entry shall be immutable; corrections occur only as compensating entries	BR-24	ledger	LedgerEntry , CompensatingEntryService	Unit + inspection — checklist item + test asserting no mutator exists
FR-18	A payment whose (bank, reference) pair is already in the ledger shall be rejected as a duplicate	BR-25	ledger	IdempotencyGuardPoster	Unit — EP : new reference / same reference twice
FR-19	A payment against a closed or written-off loan shall be rejected with a typed exception	BR-26	ledger	LoanAccount , LoanClosedException	Unit — EP : active / closed / written-off
FR-20	Repayments shall be importable from bank statement files in each supported bank's format	—	ledger.importing	HblStatementAdapter , McbStatementAdapter	Unit + integration — EP : valid row / malformed date / minor-unit amount / duplicate reference
FR-21	Penalty shall accrue at 0.05% per day on overdue principal, capped at 10% of the overdue amount	BR-27, BR-28	delinquency	PenaltyAccruer	Unit — BVA : just below / at / above the cap (unit level: under BR-32 the cap cannot be reached before write-off); EP on days overdue
FR-22	Days past due shall be counted from the day after the instalment due date	BR-29	delinquency	DaysPastDueCalculator	Unit — BVA : due date / due + 1
FR-23	Accounts shall be bucketed 0 / 1-30 / 31-60 / 61-90 / 91+ days past due and classified accordingly	BR-30	delinquency	DelinquencyClassifier , method classify()	Unit — Basis path (CC ≈ 6) + BVA : 0/1/30/31/60/61/90/91
FR-24	A provision percentage shall be applied per classification	BR-31	delinquency	ProvisioningPolicy	Unit — EP : one case per classification
FR-25	A loan more than 180 days past due shall be written off automatically	BR-32	delinquency	WriteOffPolicy	Unit — BVA : 179 / 180 / 181 days
FR-26	End-of-day processing shall be idempotent per business date	BR-33	delinquency	EndOfDayEngine , delinquency_snapshot unique key	Unit — Invariant : the same date run twice changes nothing
FR-27	A classification change shall raise an alert event	BR-34	delinquency	ClassificationChangeEvent	Unit — EP : downgrade / upgrade / no change

FR-28	The system shall produce a customer statement for a date range, with a running balance	BR-35	reporting	StatementRenderer	Unit — EP : empty range / single entry / full history
FR-29	The system shall report portfolio at risk over 30 days, provision coverage and collection efficiency	BR-36 – BR-38	reporting	PortfolioReporter	Unit — Golden file : a fixed portfolio with hand-computed metrics
FR-30	Reports shall be reproducible as at any past business date	BR-39	reporting	PortfolioReporter	Unit — EP : current date / past date / date before the first loan
FR-31	Every teller posting shall record an audit journal line naming the operator; a bank import records one line per file	—	ledger	AuditLoggingPoster (teller); the import job (per file)	Unit — EP : teller posting / imported file
FR-32	Portfolio reports shall be exportable as CSV	—	reporting	ReportExporter , CsvReportExporter	Unit — EP : empty portfolio / one loan / full portfolio

2.3 Quality requirements traced

ID	Verification method	Primary artifact (planned)	Evidence produced at
QR-01	Test	EndOfDayPerformanceTest	M2, M3
QR-02	Test	GoldenScheduleTest	M2, M3
QR-03	Test + demonstration	PostingFaultInjectionTest	M2, demonstrated M3
QR-04	Analysis	Static-analysis metric report in CI	M2, M3
QR-05	Inspection	Fagan inspection checklist item + LedgerImmutabilityTest	M2
QR-06	Test	EndOfDayConcurrencyTest over EndOfDayEngine and LockRegistry , 50 runs per scheduled CI job	M2, M3

2.4 Coverage of the matrix at M1

Measure	Status at M1
Functional requirements defined	32
Quality requirements defined	6
Constraints recorded	4
Requirements with a planned verification approach	38 of 38 (100%)
Requirements with executed tests	0 — M1 is a planning milestone
Planned design elements traceable to a requirement	Every planned business-logic class in the SE423 class views traces to an FR or QR; the rest are shared kernel and infrastructure types
Code modules traceable to a requirement	n/a at M1; audited from M2

Audit rule for M2 and M3: every requirement must have at least one test (no test gaps), and every production class must trace to at least one requirement (no rogue code). Both directions are checked, because neither direction can find the other's defect.

3 SQA Plan — Scope and Outline (IEEE 730)

The outline below follows the SQAP outline in Annex C of IEEE 730-2014. Every mandatory section is listed with a one-line plan specific to MicroLend. M1 establishes the scope and the structure; the plan is developed through M2 and completed as the final SQAP at M3 (Week 13), checked against the final build.

§	IEEE 730 section	Plan for MicroLend
1	Purpose and scope	This SQAP governs the MicroLend loan-servicing system across M1–M3, covering the five modules and the shared repository, tailored to a three-person Very Small Entity
2	Definitions and acronyms	Define the terms actually used — DPD, PAR30, allocation waterfall, EOD, compensating entry, cyclomatic complexity, EP/BVA — plus error, defect and failure, so the document is self-contained
3	Reference documents	The joint-project guidelines (updated 7 October 2026: three milestones), ISO/IEC 25010:2011, ISO/IEC/IEEE 29148, IEEE 730-2014, IEEE 1012 (V&V plan at M2), IEEE 1028, ISO/IEC 29110, the Business Rules Register and the RTM
4	SQA plan overview	Organisation, product risk, tools, standards and effort for MicroLend — 4.1 to 4.5 below
4.1	Organisation and independence	Name the three members, their module ownership and the rotating SQA lead. State that organisational independence is impossible in a three-person team, and that it is substituted by non-owner review (no one approves their own pull request) and the external inspector at M2
4.2	Software product risk	Classify MicroLend's product risk as financial, not safety-related ; identify the money path (<code>RepaymentAllocator</code> , <code>DelinquencyClassifier</code> , the ledger) as the high-risk area; set SQA depth from that, since SQA effort is proportionate to product risk
4.3	Tools	Java 21 and JUnit 5, SQLite, Gradle, GitHub and GitHub Actions, JaCoCo for coverage, Checkstyle as the coding-standard gate, ArchUnit for the no-cycles constraint, and a second static analyser for the M2 report — each named with the quality question it answers
4.4	Standards, practices and conventions	Constraints C-01 to C-04; the commit convention <code>test(red): / feat(green): / refactor: ;</code> the documentation standard; and the 100% decision coverage on the money path policy
4.5	Effort, resources and schedule	About 39 student-days of construction in total, plus about 8 student-days of SQA per milestone (24 across three), so about 63 student-days — roughly three person-months — mapped onto the Week 6 / 10 / 13 milestone dates with named owners
5	Activities, outcomes and tasks	Product assurance (5.1) and process assurance (5.2), each tied to a milestone deliverable
5.1	Product assurance	Evaluate the plans, the product and its support documentation, and measure the product — 5.1.1 to 5.1.5
5.1.1	Evaluate plans for conformance	At each milestone, check that the M1 proposal, the test plan and this SQAP still describe what is actually being built; record deviations

5.1.2	Evaluate product for conformance	The Fagan inspection of the two money-path classes, static-analysis triage, and black-box and white-box test execution against the RTM requirement IDs
5.1.3	Evaluate product for acceptability	Define the acceptance gate applied to the final integrated system at M3: all functional requirements traced to passing tests, $\geq 60\%$ coverage of core logic with 100% decision coverage on the money path, zero open critical defects
5.1.4	Evaluate product life-cycle support for conformance	Check at each baseline that the README, setup instructions, schema DDL and design documents match the code
5.1.5	Measure products	Defect density per module, branch coverage, cyclomatic complexity per method, open and closed defect counts — the metrics the GQM questions consume
5.2	Process assurance	Evaluate the life-cycle processes, environments and subcontractor processes, measure the processes, and assess staff skills — 5.2.1 to 5.2.5
5.2.1	Evaluate life-cycle processes for conformance	Verify that the red-green-refactor sequence, pull-request review and merge gates were genuinely followed, using commit and pull-request history as the evidence
5.2.2	Evaluate environments for conformance	Confirm all three members build and test on the same toolchain versions, and that CI is the single source of truth for a green build
5.2.3	Evaluate subcontractor processes for conformance	Declared not applicable, with justification: there are no subcontractors. Third-party components are limited to open-source libraries (the SQLite JDBC driver, Apache Commons CSV, and test libraries), pinned to exact versions in <code>build.gradle</code> as each is added, and checked by CI on every build. The standard requires the declaration, not silence
5.2.4	Measure processes	Inspection rate in LOC per hour, defect-removal effectiveness by phase, and rework as a percentage of total effort
5.2.5	Assess staff skill and knowledge	Record that concurrency, white-box test design, static analysis and the Fagan inspection process are not yet taught at M1, and name the self-study and pairing that closes each gap before the technique is used
6	Additional considerations	Contract review, quality measurement, waivers, task repetition, SQA risk, communication and non-conformance — 6.1 to 6.7
6.1	Contract review	Treat the joint-project guidelines as the agreement: a checklist mapping every Section 3.1 and 3.2 minimum to where it is met, re-checked at each milestone
6.2	Quality measurement	The GQM programme — one goal, three questions, and the metrics listed in §5.1.5 and §5.2.4
6.3	Waivers and deviations	Any departure from this plan is logged with reason, approver and date — for example, dropping a planned pattern is recorded, not silently abandoned
6.4	Task repetition	A re-inspection is triggered if an inspection finds more than five major defects in the roughly 250-line inspection package; the full regression suite re-runs on every push
6.5	Risk to performing SQA	Risks to the quality work itself: exam weeks compressing the inspection, the external inspector falling through, one member carrying the SQA load
6.6	Communications strategy	A weekly 15-minute session where each member explains their module's invariants to the other two; defect log and corrective actions in the repository; milestone status summarised in the M2 and M3 documents
6.7	Non-conformance process	How a defect or process non-conformance is raised, triaged by severity, assigned, corrected and verified — the corrective-action (CAPA) loop opened at M2 with the inspection and static-analysis findings, and closed — or kept open with a stated reason — in the M3 final audit
7	SQA records	What is recorded and where it is kept — 7.1 and 7.2

7.1	SQA records: collect, file, maintain	All quality records live in <code>/qa</code> in the repository: inspection log, defect log with corrective-action status, static-analysis reports, coverage reports, GQM data and risk-register versions
7.2	Availability of records	Records are versioned with the code in the hosted repository, so any record can be produced on request at the demo

3.1 Inspection planning note (for M2)

The inspection runs in six stages: **planning** → **overview (kickoff)** → **preparation (individual checking)** → **inspection (logging) meeting** → **rework (edit)** → **follow-up and exit**. The planning guideline is **100–200 lines of code per hour** for source code.

That rate is the number to plan with. Inspecting `RepaymentAllocator` plus `DelinquencyClassifier` — roughly 250 lines — means a logging meeting of about 1¼ to 2½ hours — split into two sessions of at most two hours if the slower rate applies — which is schedulable with an external inspector and which also provides the denominator for the defect-detection-rate metric in the GQM programme. No member moderates their own code: Muhammad Ibrahim moderates `RepaymentAllocator` (Tughrul Hussain's module), Hassan Khalid moderates `DelinquencyClassifier` (Muhammad Ibrahim's), and the external inspector reviews both.

4 Cost of Quality and Quality-Culture Statement

4.1 The model

Total cost of quality is the sum of the **cost of conformance** (what is spent to prevent and detect defects) and the **cost of non-conformance** (what defects cost once they exist).

4.2 Expected costs for this project, in student-hours

The currency here is time, because that is the resource the team actually spends. Figures are estimates made at M1 and compared against actuals at M3 — the comparison is itself a GQM metric.

Category	Activity for MicroLend	Estimated effort (student-hours)
Prevention	Writing the Business Rules Register and freezing scope	6
	Coding standard and Checkstyle configuration (constraints C-01 to C-04)	4
	Hand-computing 30 golden schedules in a spreadsheet before coding	8
	Design review of the class model before implementation	6
	Setting up CI, branch protection and quality gates	5
	Prevention subtotal	29
Appraisal	Writing and maintaining the test suite (test-first)	60
	Pull-request review by a non-owner	18
	Fagan inspection of the money path, including preparation	12
	Static-analysis runs and triage	8
	Appraisal subtotal	98

Internal failure	Rework of defects found by tests, review or inspection before submission	25 (expected)
External failure	Rework of defects found by the grader at a milestone demo, or by the following milestone	10 (target: as close to zero as possible)
Non-conformance subtotal		35
Total expected cost of quality		162 student-hours

4.3 What the team expects these numbers to show

Three things follow from this table, and each is a prediction the team is prepared to be held to at M3:

- 1. Appraisal dominates, and that is correct for this project.** The money path has no safety net other than tests: a wrong instalment does not crash, it quietly produces a wrong number. Detection effort is where the risk is.
- 2. Internal failure cost rising is a good sign, not a bad one.** If the register shows more hours of rework triggered by our own tests and inspections at M2 than at M1, it means defects are being caught inside the team rather than at a demo. The number to minimise is **external** failure, not internal. A team reporting zero internal failure cost is reporting that it is not looking.
- 3. The 1:10:100 principle is why the 8 hours on golden schedules are spent first.** A rounding rule corrected while writing the spreadsheet costs minutes. The same defect found after schedules, ledger entries, delinquency buckets and portfolio reports have all been built on it requires reworking every one of them, under deadline. Prevention is bought at the cheapest point on the curve.

4.4 Quality-culture statement

The team commits to four behaviours, each with a mechanism that makes it checkable rather than aspirational:

Commitment	The mechanism that enforces it
Quality is not a phase at the end. Quality activities run in the same milestone cycle as the construction work they assess	SQA deliverables are owned per milestone and rotate; the M2 inspection targets M2 code, not a frozen artifact
No one marks their own homework. Every change is reviewed by someone who does not own that module	Branch protection requires one approving review from a non-author; <code>CODEOWNERS</code> routes the request
Defects are data, not blame. A defect found in inspection is a success for the process	The defect log records type, severity and phase found — never who wrote it. The metric is defect-removal effectiveness, not individual defect counts
If it is not evidenced, it did not happen. Every claim in this plan maps to an artifact in the repository	All quality records live in <code>/qa</code> , versioned with the code and reproducible from a milestone tag

5 Standards Awareness

ISO/IEC/IEEE 12207 tells us *what processes must exist*. Of its processes we genuinely perform Configuration Management (branch protection, the `m1` – `m3` baseline tags) and Quality Assurance, which the 2017 edition groups as technical management processes, and Verification and Validation, two separate technical processes; our pull-request reviews and the Fagan inspection correspond to the 2008 edition's Software Review process. **IEEE 730-2014** tells us *how* the SQA function executes inside that lifecycle: its activities in three groups — SQA process implementation, product assurance, process assurance — its principle that the depth of SQA is set by **product risk**, and its rule that any activity not performed must be declared *not applicable with a justification* rather than quietly dropped. That rule is why our SQAP §5.2.3 explicitly marks subcontractor evaluation as not applicable instead of omitting it.

Full 12207 and CMMI adoption would be wrong at our scale. Three students on one project is a **Very Small Entity** under ISO/IEC 29110's 25-person definition, and at roughly three person-months of effort (\$4.5 of the SQAP outline) we fall within the **Entry** profile's six-person-month ceiling, though we follow the **Basic** profile's structure as taught. Its two processes map onto our milestone loop: Project Management covers planning, the risk register and milestone reviews, and Software Implementation covers the construction work each milestone delivers. Its Deployment Packages supply ready-made templates and traceability tables so our time goes into the work rather than into inventing document formats.

Against **CMMI-DEV v1.3** we claim **Maturity Level 2 behaviours only**, and we name them: Requirements Management through the RTM, Configuration Management through protected branches and baselines, Process and Product Quality Assurance through the SQAP §5.2.1 process-conformance checks and the M3 audit, and Measurement and Analysis through the GQM programme. We explicitly do **not** claim Level 3, because Level 3 requires an organisational set of standard processes, which a single student project does not have (the Fagan inspection is a peer review, which CMMI places in Verification at Level 3), and the statistical process control of Levels 4 and 5 is meaningless with three people and one data point per milestone.

MicroLend is not safety-critical, so no domain safety standard applies. We borrow one idea from them — criticality dictates technique — as the SQAP policy of **100% decision coverage on the money path** (`RepaymentAllocator` , `DelinquencyClassifier`), alongside the ban on `double` for money.

Not claimed: CMMI Level 3 or above; conformance to any safety-critical standard; organisational independence of the QA function; and certification of any kind. Every standard named above is used as a source of practice, not as a badge.

6 Initial Risk Register

The full register is maintained as a living document at `qa/risk-register.md` in the repository, updated at every milestone and whenever a risk materialises or closes. It is reproduced here as at M1.

6.1 Scoring

Risk exposure is **computed, not asserted**: $E = P \times C$, where P is probability (1 = rare ... 5 = almost certain) and C is consequence, i.e. impact (1 = negligible ... 5 = severe).

Exposure	Priority	Response
17–25	Critical	Act now; mitigation owned and tracked weekly
10–16	High	Mitigation planned and started this milestone
5–9	Medium	Monitor; mitigate if exposure rises
1–4	Low	Accept; review each milestone

6.2 The register at M1

P = probability, C = consequence, E = exposure. Residual exposure is recorded from M2, once each mitigation has produced evidence.

ID	Risk (condition → consequence)	Type	$P \times C$ = E	Priority	Mitigation	Owner	Status
R1	Money or date arithmetic in schedule generation or the allocation waterfall is subtly wrong → every downstream figure is wrong and the defect is found late	Product	4×5 = 20	Critical	Hand-compute 30 golden schedules before coding and commit them as the first tests; <code>BigDecimal</code> only, enforced by constraint C-01; property test that Σ instalments = principal + total interest	Hassan Khalid	Open
R2	M1 work starts late → the compressed schedule pushes work into the M2 window, and M2 (Week 10) carries 35%	Project	5×3 = 15	High	Freeze scope at five modules with no new features; draft M1 documents from the existing analysis; set up repository and tooling in parallel with document writing	Tughrul Hussain	Open
R3	The external inspector required at M2 is not arranged in time → the inspection criterion (20% of SE431 M2) cannot be fully evidenced	Process	3×4 = 12	High	Request the instructor's team pairing immediately after M1; agree a date in Week 8; prepare the inspection package in advance	Muhammad Ibrahim	Open
R4	Commits are squashed or tests are written after the code → the red-green-refactor trail cannot be reconstructed, losing TDD marks at M2 (10%) and M3 (15%)	Process	3×4 = 12	High	Squash-merge disabled in repository settings from day one; commit convention enforced; merges use <code>--no-ff</code> ; the pull-request template asks for the red commit hash	Tughrul Hussain	Open
R5	Concurrency defects in the end-of-day engine appear only under load → late, hard-to-reproduce failures near the M3 concurrency-correctness check	Product	3×4 = 12	High	Immutable value objects; per-account locks, one held at a time; a unique <code>(loan_id, business_date)</code> idempotency key in the same transaction as accrual; deterministic latch-driven tests with no <code>sleep</code>	Muhammad Ibrahim	Open
R6	Individual contribution is not attributable → individual evidence suffers, which is 20% of each member's milestone mark	Project	3×3 = 9	Medium	Module ownership equals directory ownership; every change by pull request reviewed by a non-owner; SE431 deliverable ownership rotates each milestone; AI Usage Log maintained continuously	Tughrul Hussain	Open
R7	Several M2-graded techniques are not taught until after they must be designed for → the team designs ahead of the lectures and gets it wrong	Project	4×4 = 16	High	Design the structures now, defer detailed technique work until taught; start the evidence streams that cannot be reconstructed later at M1; re-check each design against the lecture when delivered	Muhammad Ibrahim	Open
R8	AI-assisted work is submitted that a member cannot explain in the viva → reduced or zero individual marks, and a possible integrity concern	Project	3×5 = 15	High	AI Usage Log updated continuously; the module owner reviews and rewrites generated code in their own module; dry-run Q&A inside the team before each demo	Hassan Khalid	Open

6.3 Review schedule

- **M1:** register initiated with eight risks, scored and owned.
- **M2 (Week 10):** every risk re-scored using evidence from the architecture, inspection, static analysis, testing and SCM; risks arising from implementation added; residual exposure recorded.
- **M3 (Week 13):** continuous updating from M1 to M3 demonstrated; mitigation effectiveness evaluated against actual project evidence; residual risk recorded for every risk.

7 Summary of M1 Quality Position

Deliverable	Status at M1
Quality requirements	6 defined, each with a measurable target and a named verification method; all eight characteristics rated, two of them Low with justification
Constraints	4 recorded, C-01 enforced by the build
RTM v1	32 functional and 6 quality requirements, each traced to a business rule where one exists and given a planned verification approach and a named test design technique
SQAP	Scope set and all mandatory IEEE 730 sections outlined with project-specific plans; one section declared not applicable with justification
Cost of Quality	Estimated at 162 student-hours with the prevention/appraisal/failure split and three stated predictions
Standards awareness	12207, IEEE 730, 29110 and CMMI-DEV related to this project's scale, with explicit non-claims
Risk register	8 risks, computed exposure, owners assigned, living document in the repository

What M1 deliberately does not yet contain, because the techniques are taught later in the semester and M1 does not require them: executed test results, the inspection defect log, the static-analysis report, coverage figures and GQM data. Each has a planned home in this document and an owner. Of the four evidence streams that cannot be reconstructed afterwards, three are already running at M1 — the TDD commit trail, the SCM baselines and the CI static-analysis reports (kept for 90 days) — and GQM data collection starts with the first feature branch at M2.

Annex A — AI Usage Log

Required by the joint-project guidelines §9. Milestones 2 and 3 carry their own logs.

Every entry used Claude through Claude Code. Each entry is dated the day the work was done, which the repository's commit history corroborates.

#	Date	Tool	Purpose	Prompt / activity summary	Material adopted	Modifications made	Verification
1	2026-10-04	Claude Opus 5	Compare candidate project ideas against both rubrics	Asked for candidate project ideas scored against the §3.1 and §3.2 minimum requirements	Comparative analysis; MicroLend chosen over two alternatives	Scope cut to a locally-run application with no deployment or paid services	Each rubric minimum traced to where it is met; team accepted the choice
2	2026-10-04	Claude Opus 5	Draft the SE423 document	Asked for a class model, SOLID analysis and pattern justifications from the module list and business rules	Class model, SOLID analysis, pattern justifications	Diagram split into views for legibility; CLI replaced by a local web interface	Found and corrected a Simple Factory presented as a Factory Method
3	2026-10-04	Claude Opus 5	Draft the SE431 document	Asked for ISO 25010 quality requirements, an RTM, an IEEE 730 outline, a CoQ estimate and a risk register for the SE423 design	Quality requirements, RTM, SQAP outline, CoQ, risk register	Four items reclassified from quality requirements to constraints	Targets and effort estimates checked as realistic for a three-person team
4	2026-10-04	Claude Opus 5	Repository setup and coding standard	Asked for a Checkstyle configuration enforcing C-01, a CODEOWNERS file, a PR template and branch rules	Checkstyle config, CODEOWNERS, PR template, branch rules	C-01 check rewritten as a token rule after a false positive	Branch protection tested by a direct push, which was blocked and logged
5	2026-10-04	Claude Opus 5	Business Rules Register	Asked to turn each module's rules into numbered rules with boundary values for test design	43 numbered rules with boundary values	Numeric thresholds reviewed and accepted unchanged by the team	Boundary values cross-checked against the RTM test design

6	2026-10-04	Claude Opus 5	Build and CI scaffold	Asked for a Gradle build, a failing <code>MoneyTest</code> first, then <code>Money</code> , and a GitHub Actions workflow	Gradle build, <code>Money</code> , <code>MoneyTest</code> , GitHub Actions workflow	Written as a real TDD cycle, red commit before green	CI green: compile, 11 tests, Checkstyle, coverage
7	2026-10-04	Claude Opus 5	Submission PDFs	Asked for a pandoc/LaTeX script producing both submission PDFs with a title page	Build script and title page	Fonts changed to carry mathematical symbols	Pages inspected; a fault dropping <code>≤</code> and <code>×</code> was found and fixed
8	2026-10-07	Claude Opus 5.5	Pre-submission review of both documents and the repository	Asked to check both M1 documents against the joint guidelines and rubrics, list inconsistencies, then apply the fixes	Corrections to both documents, RTM, risk register, business rules, README and CODEOWNERS: M1 Team Lead, class views matched to the text and RTM, SOLID status labels, milestone references matched to the updated guidelines, an independent fact-check pass, PDFs rebuilt with an HTML/CSS pipeline (Paged.js)	M1 Team Lead confirmed by the team; an interim four-milestone revision was reverted to three milestones when the instructor issued the updated guidelines the same day; risk-grade assignment and active-loan counting defined at the team lead's request and confirmed by all three members (BR-03, BR-09)	Each change checked against the guidelines PDF and the live repository (ruleset, CI runs, <code>Money.java</code>); PDFs rebuilt and every page inspected

Declaration for Milestone 1

Each member named below has read, understood and verified the material credited to them, can explain and defend the design decisions, quality requirements and analysis it contains, and reviewed and modified any AI-assisted content where needed.

- **Muhammad Ibrahim — 2023446** · `origination`, `delinquency`; SQAP outline, risk register
- **Hassan Khalid — 2023242** · `products`, `reporting`; quality requirements, RTM
- **Tughrul Hussain — 2023532** · `ledger`; standards awareness, Cost of Quality, repository setup

Team rules for AI use

1. Generated code enters the repository only through the module owner, who reviews it and answers for it.
2. No generated test is accepted without reading what it asserts.
3. Business rules and numeric thresholds are the team's decisions, not defaults.
4. Entries are added when the work happens, not before a deadline.
5. Anything a member cannot explain is rewritten or withdrawn before submission.